## Chapter Three: Relational Data Model

### 3.1. Introduction

The relational model was proposed by Dr.Codd in his mathematical paper of 1970s as an attempt to simplify the database structure. The model eliminates the parent/child structures from database and in return present database as simple row/column tables of data values.

In Relational database, all the data visible to the user is organized strictly as tables of data items and where all operations work on these tables. Based on this definition, we can argue out that a relational model is based on a mathematical concept of a relation which is physically represented as a table Relational data model usually represent data logically as collection of relations (tables). Each relation (table) has a name and is made up of named attributes (column) of data. Each tuple (row) contains one value of attribute.

Relational DBMS usually represent parent/child relationship using data values contained in the databases.

A number of several commercial products exist in the market which uses relational data models. Examples include oracle, MS SQL server, SYBASE, INFORMIX e.t.c. several free open source products include MYSQL, PostgreSQL. Relational data model is the most dominant for developing database applications. Some SQL relational standards include SQL-89 (SQL1), SQL-92(SQL2) SQL-99 e.t.c

### 3.2. Basic terminologies for relational data model

#### 3.2.1 Tables/relation

It's a set of data elements (values) organized as refers as rows and columns to a table with rows and columns. A RDBMS requires database to be perceived by user as tables and has the following properties

1. Each attribute has a distinct name.
2. A relation has a distinct name from the other relations.
3. Each cell of the relation contains exactly one atomic (single) value
4. Each tuple is distinct i.e. there are no duplicate tuples.
5. The order of the attributes and tuples has no significance.

6. The values of an attribute are from the same domain.

### 3.2.2 Attribute/column/field

It's a named column of a relation. In relational model, relations are used to hold information about objects to be represented in the database. Relations are represented as a two-dimensional table in which rows of the table correspond to individual records and tables columns correspond to attributes.

### 3.2.3 Domain

It's a set of allowable values for one or more attributes. Each attribute of a relation is defined on a domain. They may be distinct for each attribute or more attributes maybe defined on the same domain. For example, if we have an attribute named gender, its domain values would be male and female

### 3.2.4 Tuple

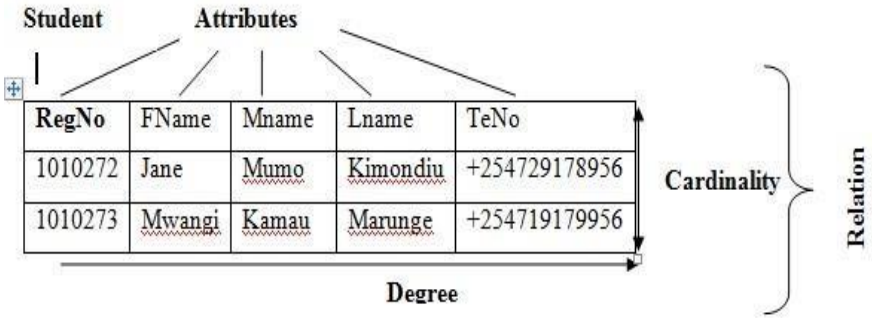
This is a row of a relation. They can appear in any order and the relation will still convey the same meaning.

### 3.2.5 Degree of a relation

This refers to the number of attribute a given relation contains. A relation with one attribute will have degree one and will be referred to as unary relation, relation with two attributes is usually known as binary, one with three is referred to as tertiary and so on

### 3.2.6 Cardinality of a relation

It refers to the number of tuples a relation contains. Cardinality changes as tuples are deleted and added to a relation. It's a property of the extension of a relation




## 3.3. Database Keys

Database keys are an integral component of database design since they ensure the integrity of the data contained and retrieved from the database. Moreover, they ensure each record within a table can be uniquely identified by one

or a combination of fields within the table as well as help identify the relationship between tables.

The following section discuss of the database key

**3.3.1 Super key**

This refers an attribute/set of attributes that uniquely identifies a tuple within a given relation. It may contain additional attributes that are not necessary for unique identifications.

**3.3.2 Candidate Key**

It's an attribute/set of attributes that has possibility of being nominated as primary key. For an attribute to be nominated as primary key, it has to meet the following

- It must contain unique values to uniquely tuples in a record
- It must not contain null values
- It contains the minimum number of fields to ensure uniqueness

**3.3.3 Primary key**

Once your candidate keys have been identified you can now select one to be your primary key. A primary key refers to a candidate key that has been selected to uniquely/chosen to uniquely identify tuples within a relation. The key ensures tuples do not duplicate in a relation. Just like a candidate key, a primary key must

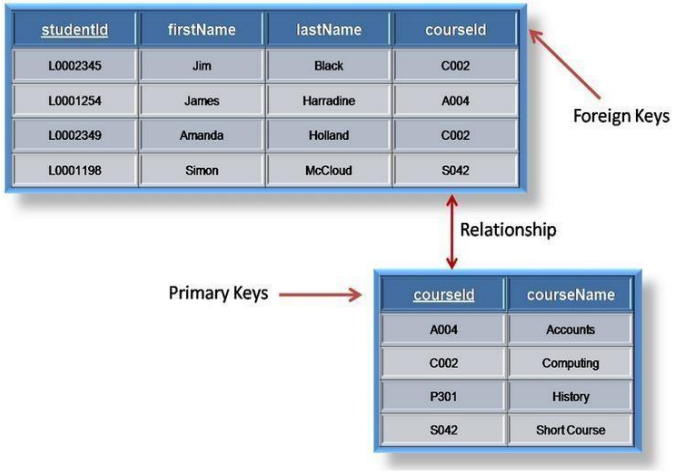
- contain unique values,
- Never be null a
- Uniquely identify each record in the table.

**3.3.4 Foreign key**

A foreign key is generally a primary key from one table that appears as a field in another where the first table has a relationship to the second i.e. if we had a table A with a primary key X that linked to a table B where X was a field in B, then X would be a foreign key in B.

Let's consider two tables i.e. student and course table. The course contains a course_id which uniquely identifies each course hence the primary key of the table course. On the hand, the course table contains the course_id which denotes which course a particular student does. The 2 tables are

linked through course_id and as such course_id would be a foreign key in the student table.




### 3.3.5 Compound Key

It's a key consisting of more than one field to uniquely identify a record. Let's Assume we have an entity named enrolment, which holds the courses on which a student is enrolled in a given semesters. A student is allowed to enroll on more than one course and a course can be enrolled by zero or many students. We can represent the information as follows

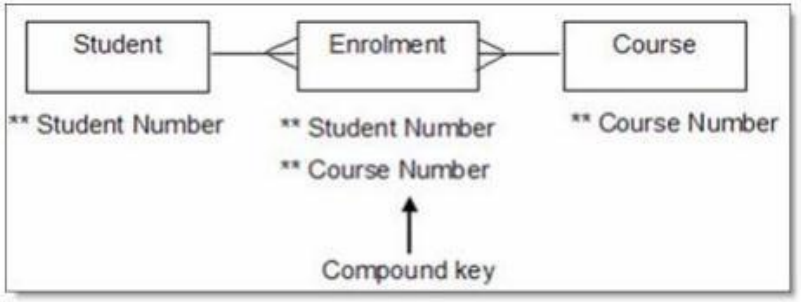
**3.3.6 Secondary Key or Alternative Key** When a table may have more than one candidate key and one has been selected to be the primary key, the rest become secondary/alternate keys.

### 3.3.7 Simple Key

A simple key is a key consisting of a single attribute to uniquely identify an entity occurrence. In addition, the field in itself cannot be broken down into other fields e.g. student_id, which uniquely identifies a particular student, is a single field and therefore is a simple key. No two students would have the same student_id.

### 3.3.8 Compound key

It's a key consisting of more than one field to uniquely identify a record. Let's Assume we have an entity named enrolment, which holds the courses on which a student is enrolled in a given semesters. A student is allowed to enroll on more than one course and a course can be enrolled by zero or many students. We can represent the information as follows




Student number and course number combined which are primary key for student and course respectively can be treated as compound key in enrolment table since the two collectively can uniquely identify all the record for the table.

### 3.3.9 Composite key:

On the other hand, a composite key consists of more than one attribute to uniquely identify an entity occurrence. This differs from a compound key in that one or more of the attributes, which make up the key, are not simple keys in their own right.

Let's assume we decide to uniquely identify all the record using two attributes i.e. student number + lastName while we uniquely identify the course using the course number. Since student number and LastName are not each simple keys, they have to be combined in order to uniquely identify the student hence they can be referred to as composite key.

## 3.4. Integrity Constraints

As noted earlier in our previous chapter, one component of the data model is a ***set of constraints*** to ensure data is executed accurately.

Integrity is a major component of the relational model since it ensures data values in the database are correct and consistent using entity and referential integrity rules.

There are three major types of integrity constraints that can be implemented by a relational Database Management Systems (RDBMS) i.e. entity, referential and general constraints

### 3.4.1 Entity integrity

This restriction applies to primary key of base relation. The rule states that for every instance of an entity, the **value of the primary key must exist**, be u**nique,** and cannot **be null.**

Without entity integrity, the primary key could not fulfil its role of uniquely identifying each instance of an entity.

### 3.4.2 Referential Integrity

This restriction deals with foreign key and it states that every **foreign key value must match a primary key value** in an associated table or **the value is wholly null**. Referential integrity ensures that we can correctly navigate between related entities.

### 3.4.3 General constraints

These are additional rules/restrictions specified by users/DBA that specify additional constraints that data must satisfy with respect to a particular enterprise. The DBA can specify a rule that salary of staff should not be less than 5000, which the BMS will enforce.

## 3.5. Insert and Delete Rules

A foreign key creates a hierarchical relationship between two associated entities. The entity containing the foreign key is known as **child/dependent** table while the table containing the primary key from which the foreign key values are obtained is known as **parent.**

In order to maintain referential integrity between the parent and child as data is inserted or deleted from the database certain insert and delete rules must be considered.

### 3.5.1 Insert Rules

Insert rules commonly implemented are:

- Dependent insert rule

  This rule allows insertion of a foreign key value only if matching parent entity instance/primary already exists.

- Automatic insert rule

  This rule allows insertion of a foreign key value. If matching parent entity instance/primary key value does not exist, it is created.

- Nullify insert rule

  This rule which allows insertion of a foreign key value. If a matching parent entity instance/primary does not exist, the foreign key in child is set to null.

- **Default insert rule**

  This rule which allows insertion of foreign key value .If a matching parent entity instance/primary key value doesn't exist, the foreign key in the child is set to previously defined value.

- **Customized insert rule**

  This rule allows insertion of a foreign key value only if that value meets certain customized validity constraints

- **No Effect insert rule**

  This rule allows insertion of a foreign key value without doing a validity check to ascertain if a matching primary key value exists or not in associated dependent table

### 3.5.2 Delete Rules

**The following delete rules can specified while creating a database**

- **Restrict delete rule**

  This rule allows deletion of parent entity instance/primary key value only if there are no matching child entity instances.

- **Cascade delete rule**

  This rule always permits deletion of a parent entity instance/primary key value and deletes all matching instances in the child entity.

- **Nullify delete rule**

  This rule permits deletion of a parent entity instance/primary key value. If there any matching child entity instances exist, their values are set to null.

- **Default delete rule**

  This rule allows deletion of parent entity instance/primary key value. If any matching child entity instances exist, their values are set to a predefined default value.

- **Customized delete rule**

  This rule allows a primary key value to be deleted if it follows a certain validity constraint are met.

- **No Effect delete rules**

  This rule allows deletion of primary key value without doing a validity rule to ascertain if a matching child entity instance exist or

### 3.6. Domains

Each attribute/column in the model should be assigned domain information which includes:

- **Data Type**—Basic data types are integer, decimal, or character. Most databases support variants of these plus special data types for date and time.
- **Length**—this is the number of digits or characters in the value.
- **Date Format**—the format for date values such as dd/mm/yy or yy/mm/dd
- **Range**—the range specifies the lower and upper boundaries of the values the attribute may legally have
- **Constraints**—are special restrictions on allowable values.
- **Null support**—Indicates whether the attribute can have null values
- **Default value (if any)—**the value an attribute instance will have if a value is not entered.

### 3.7. Base relations

Named relations corresponding to an entity in the conceptual schema whose tuples are physically implemented.

### 3.8. Chapter summary

SQL is based on relational data model which organizes data in a database as collection of columns

- Each table/relation has a unique name that identifies it
- Each table has one or more columns (field/attributes) arranged in a specific left to right order.
- Each table has zero or more rows (tuples) each containing a single data devalue in each column. Rows can be arranged in any order.
- All data values in a given column have the same data type are from a given domain of values called domain of a column

Tables are related to one another by the data they contain. Relational data model uses the database keys to model these relationships among the table.

- A primary key is a field or combination of fields that uniquely each tuple of a table
- A foreign key on the other hand is a field or combination of fields in tables whose values are primary key value for some other table. A table can have one or more than one foreign key linking it to one or more tables.
- A primary/foreign key combination usually creates a parent/child relationship between the tables themselves

## 3.9. Chapter exercise 3.10. More reading references

1. James, R. Groff & Paul, P. (2006). The complete reference SQL (2rd ed.). Oxford, India: Tata McGraw-Hill publishing Company Limited. [Pp57-67]